

Want to Write Quality Code? Try Logging in Python

Tired of spending long hours in debugging your code? The logging system in Python is designed to help you keep an eye on these bugs and improve the quality of your code substantially. Let's see how it works.



Logging is an important part of software development and helps reduce endless hours of debugging. It can be used for different flow analyses, health checks, issue analysis, performance checks, security scans, audit and so on, if used correctly.

Log calls from the program indicate that certain events have occurred during the timestamp, may be from the IP address, and also give some additional information. The events hold the descriptive message — for example, dynamic or static values, importance of message (debug, info, warn, error), etc.

Python provides a logging system as a part of its standard library, making it easier for you to add it to your application.

Logging module

The logging module that helps add logs is easy and readily available, as shown below:

```
import logging
```

```
logging.debug('This is a debug message')
logging.info('This is an info message')
logging.warning('This is a warning message')
logging.error('This is an error message')
logging.critical('This is a critical message')
```

The output is:

```
WARNING:root:This is a warning message
ERROR:root:This is an error message
CRITICAL:root:This is a critical message
```

As seen from the code, there are five levels of logging, which indicate the severity of events. Each level has a corresponding severity associated. The levels defined in Table 1 are based on the severity in increasing order.

Level	Used for
Debug	Detailed messages, usually for diagnosing problems
Info	Confirmation or facts of working as expected
Warning	Alarming for future breaks, but until now the application is working
Error	Due to a serious problem in the function
Critical	Due to a serious problem, the program failed to continue to execute

Table 1: Usage of each log level